Prosperity 4 Writeup

TLDR:

My main contribution was **strategy direction and market-structure discovery**. I was not just tuning parameters. I was pushing the team toward new hypotheses: “what is the hidden structure here, and what strategy basin should we search in?”

The clearest example was **Round 2**, where my OSMIUM handler changed the plan completely. It was not a small tweak to the existing strategy. It introduced a different architecture around rolling fair value, book imbalance, fair-gap alpha, inventory skew, and time/heat behavior. That mechanism became the foundation of the shipped R2 strategy and survived multiple audits and baseline comparisons.

Dennis’s main contribution was **optimization, verification, and execution discipline**. He helped take my strategy ideas and pressure-test them through backtests, portal checks, replay harnesses, parameter sweeps, ablations, cleanup, and ship decisions. Basically, I pushed the search into new regions; Dennis helped prove which parts actually survived reality. Rude of reality to require evidence, but here we are.

Across the tournament, my value was mostly in:

- spotting market structure before defaulting to prediction;
- proposing mechanisms that opened new strategy directions;
- pushing clue/relationship-based thinking in the voucher rounds;
- treating products as linked systems rather than isolated tickers;
- forcing the question: “what constraint is this market secretly obeying?”

The team’s best loop was:

idea → implementation → backtest → audit → ship → postmortem

My role was strongest at the **idea and market-structure stage**. Dennis was strongest at the **optimization and verification stage**. The final results came from the interaction between both: I generated direction, Dennis grounded it, and the team converted the best surviving ideas into submissions.

I’m Moses. For Prosperity 4, I worked with a team that had real quant experience, and I learned a lot from them. My main role was basically algorithm direction. I was the person pushing the strategy ideas, market reads, and weird mechanism hypotheses. Dennis was the teammate who helped optimize, verify, and ground those ideas through out-of-sample backtests, portal checks, and actual execution logic.

The best way to describe the whole competition is that it became a loop: I pushed strategy direction, Dennis helped test and optimize it, and the work improved through repeated backtesting, auditing, and post-round analysis. I was not mainly just changing parameters or making tiny tweaks. My biggest value was opening new search spaces: spotting structure, proposing mechanisms, and asking whether the market was obeying some hidden rule that the current strategy was missing.

That became clearest in Round 2. Before my OSMIUM run, the plan was to keep optimizing around the existing OSMIUM handler. Then my portal run 327370 came in at +9114 on the 100k preview surface, with OSMIUM alone around +643 better than the prior lock. That was too big to dismiss as noise. The old plan had to pause because my strategy suggested a structurally different way to trade OSMIUM.

That was basically how the team dynamic worked. I would bring in a market-structure idea or a strategy that didn't fit the current model. Dennis and the tooling would then pressure-test it: backtests, replay checks, portal comparisons, parameter sweeps, and ship gates. Some ideas died. Some got cleaned up. Some became the final submission. The good ideas were not good because they sounded clever. They were good because they survived testing.

Round 1: Calibration

Round 1 was the foundation round. The products were ASH_COATED_OSMIUM and INTARIAN_PEPPER_ROOT.

INTARIAN_PEPPER_ROOT had the obvious structure: it drifted upward in a very stable way, roughly +1000 per day. Once we saw that, the correct move was not to overcomplicate it. The PEPPER strategy became drift-carry plus market making. The idea was to ramp up inventory early, stay close to max inventory, buy and sell around fair value when the book allowed it, and avoid unwinding at the close because the scoring rewarded holding the drift.

The real optimization was in the knobs: how fast to ramp, how much minimum inventory to maintain, and how much spread capture could be added without losing the drift exposure. One important lesson was that MIN_INV = 70 worked better live than tighter inventory settings.

ASH_COATED_OSMIUM was more of a normal market-making product. It traded around 10000, so the early strategy used wall-mid fair value, penny improvement, and inventory skew. Round 1 also exposed a bug where OSMIUM spent too much time in degraded mode because an anchor-drift guard was too tight. PEPPER carried the submission anyway, but the lesson was obvious: one broken product handler should not be able to drag down the whole strategy.

The bigger Round 1 lesson was methodological. We learned to split PnL by product, avoid tuning only on combined score, use portal A/B testing instead of trusting local backtests blindly, separate drift-carry from market-making PnL, and prefer robust parameter regions over sharp peaks. Round 1 was not the flashiest round, but it built the base layer.

Round 2: The OSMIUM Breakthrough

Round 2 was where my role became much clearer.

The team already had an optimization path for OSMIUM, but then my strategy came in. My run used logic from 323158_ash_only.py, and the portal result made it obvious that this was not just a small improvement. It changed the search space.

The important part was that my OSMIUM handler was not just a parameter tweak. It used a different architecture: rolling-median fair value instead of a hard anchor, fair-gap alpha, microprice and book-imbalance signals, trade-flow logic, conflict dampening, heat/time-of-day profiles, inventory skew, quote-age widening, and terminal flattening.

Not every component ended up mattering. That is the whole point of testing. Some parts survived, some parts were disabled, and some turned out to be dead code. But the broader mechanism was strong enough to become the foundation of the Round 2 submission.

Once my OSMIUM mechanism became the live candidate, it was tested against the existing baselines. The Moses strategy beat combined_r2_lift by about +801 per log. The no-winddown shipped variant beat

combined_r2_lift by about +935. The shipped Moses variant beat the current G lock by about +700. The important part is that it kept winning under replay and audit, not just in one lucky run.

Then the strategy got cleaned. Quote-age penalty was disabled because it hurt fills. Terminal winddown was disabled for the 100k portal window. Order sizing and cap-relief were tested. Dead paths were removed. The final ship file became much smaller while preserving the behavior that actually mattered.

The audits also challenged my original defaults. A 12-arm mechanism audit found no KEEP-FLIPPED winners. Trade-flow constants turned out to be useless because buyer and seller fields were both populated. Restoring the old terminal flatten was terrible. Ten regime-tilt variants all failed against the Moses baseline.

That was the real validation. My first version was not perfect, but the core mechanism survived the optimizer trying to break it.

The final Round 2 ship was r2_shipped_moses.py. Portal run 362989 returned:

ASH_COATED_OSMIUM: +17539.17
INTARIAN_PEPPER_ROOT: +84024.00
Total: +101563.17

PEPPER still carried most of the round because the drift was so powerful, but my OSMIUM handler made the OSMIUM side much more serious.

There was also a painful lesson. A later wall-FV superset added another take layer on top of my base and backtested around +1242 per day better, with PEPPER unchanged and all tested days positive. By our own standards, it probably should have shipped. It did not.

That taught one of the most important operational lessons of the whole competition: alpha only counts if the exact version ships. Having a better file somewhere does not matter if it does not reach the portal. There is a big difference between finding edge and operationally capturing it.

Round 2 also produced a major infrastructure lesson: InterceptFill was the correct local fill model. The older fill model under-filled because it missed how passive orders could intercept late-taker flow. Once we reconciled against portal logs, InterceptFill matched portal behavior much better. That made later local backtests and counterfactuals more trustworthy.

Round 3: Clue Reading to Path Optimization

Round 3 started like a puzzle and became an optimization problem.

The first clue was not normal market data. The file La_trahison_des_images.png was not actually a PNG. It was JPEG data with text appended after the EOF marker: “It’s dangerous to go alone! Take this: Link.” We interpreted that as a hint: do not trade the voucher alone, trade the link.

That pushed us toward looking at relationships between the products rather than treating them independently. The products were HYDROGEL_PACK, VELVETFRUIT_EXTRACT, and a chain of VEV_* vouchers. We initially treated the vouchers like call options on VELVETFRUIT_EXTRACT, so we checked deep-ITM parity, adjacent call spreads, butterflies, convexity relationships, hidden mirror fills, and sweep events across the voucher strip.

Some of that was real, but most of it was too small after execution costs. Deep-ITM parity worked, but not enough. Hidden mirror fills existed, but were not large enough. Smile residuals and vertical arbitrage looked nice theoretically, but spreads, queue, depth, and inventory limits killed a lot of the edge.

The turning point was realizing that the portal kept using the same slice: timestamps 0..99900, 1000 ticks, and no book mismatch versus public day 2. That changed the problem completely. It was no longer "build a general model for unknown data." It became "maximize PnL on a known 1000-tick path under position constraints."

Once we knew that, the best strategy was not an elegant options model. It was path optimization. We used timed waves across HGP, VEX, and the VEV strip, then improved that into dynamic programming over the known book path. Passive/intercept-inclusive DP produced V8.

V8 became the Round 3 ceiling candidate. Local replay was around +155022, and portal validation was around +154873.

The main lesson from Round 3 was that the edge was not one magical option-pricing insight. It was known-slice liquidity capture plus position-constrained path optimization. The clue helped us find the structure, but the money came from solving the execution problem properly.

Round 4: Too Many Alphas

Round 4 had a lot of possible signals. The feature work covered microprice, order-book imbalance, deep-ITM synthetic gaps, counterparty behavior, VELVET-to-VEV lead-lag, option-surface shifts, Hydrogel mean reversion, butterfly signals, IV/moneyness features, tail-event locks, and peak protection.

The problem was not a lack of ideas. The problem was that too many ideas can make the strategy worse. A large alpha inventory is not the same as a good submission. Sometimes it is just a clean-looking way to overfit.

The high-open crash path was one useful example. Strategy 518645 used a high-open selector on VEV_5200 and shorted a crash basket. The optimized version widened the basket and entered earlier. The baseline made +13629.0 total. The optimized crash version made +46041.5. The hybrid version made +48146.5.

That looked good, but there was still execution risk. The improvement depended heavily on new legs, and portal fills could differ from local replay if routing changed. Round 4 kept reinforcing that local replay is only a model. It is useful, but it is not reality.

Another major lesson was portal horizon mismatch. One submitted strategy used timestamp gates around 350000 and 900000, but the portal run ended around 99900, so the exit logic could not fire. The fix was to avoid brittle clock-based exits and use current-state de-risking instead.

The best local benchmark ended up being 533998_velvet_vev5500_combined_upload.py, with:

Day 1: +34819.50
Day 2: +18589.00
Day 3: +50556.50
Total: +103965.00

The important part was what we rejected. The all-missed-alpha sibling was worse. Hydrogel z-reversion was marginal and gave back too much on day 3. Butterfly hurt replay. Peak/all toggles damaged day 3. Passive deep-ITM overlays added no fills.

Round 4's lesson was restraint. Having more signals does not automatically mean having a better strategy. The final file has to survive execution. It cannot just be a collection of every clever idea we found.

Round 5: Structure Beat ML

Round 5 had 50 products, 10 families, and three days of price and trade data. The winning direction was not ML. It was structure.

The biggest discovery was the PEBBLES identity:

PEBBLES_L + PEBBLES_M + PEBBLES_S + PEBBLES_XL + PEBBLES_XS ≈ 50000

The residual had a standard deviation of about 2.80 on a 50000 base. That is basically a valuation oracle. The right approach was to use the identity for fair-value quoting and basket residual reversion, not blindly cross all five legs.

SNACKPACK also had a strong structure. CHOCOLATE versus VANILLA had around -0.916 return correlation. PISTACHIO versus STRAWBERRY had around +0.913. RASPBERRY versus STRAWBERRY had around -0.924. That made SNACKPACK a pair-trading and cointegration candidate, although drift meant rolling fair values were safer than static global means.

We also tested the ML path. CNN/GNN/TFT-style models were built and tested, but they failed. Three-class accuracy was below random. Binary accuracy was basically random. The information coefficient was negligible. So the correct decision was simple: do not deploy the neural model.

That was an important lesson. A model being sophisticated does not matter if it does not pass the gate. There is no prize for architecture cosplay.

The useful overlay was actually simpler: parametric market making. PPO and RL did not work well, but a deterministic quote-placement rule did:

bid = mid - spread_factor * half_spread + skew * inventory
ask = mid + spread_factor * half_spread + skew * inventory

The best reported configuration used spread_factor = 0.54, skew = 0.18, and size = 10, with about +74414 on ROBOT_DISHES.

The lesson was that alpha is not always prediction. Sometimes it is placement. In a lot of products, the edge came from quoting correctly around fair value rather than trying to forecast direction.

The final Round 5 direction was clear: use PEBBLES basket logic as the main alpha, SNACKPACK pair logic as the secondary alpha, apply parametric market making only where the structure supported it, avoid or defensive-quote bad products, and do not deploy ML just because we built it.

What Actually Worked

Across the whole tournament, the same patterns kept showing up.

The first pattern was structure before prediction. PEPPER drift, PEBBLES identity, voucher parity, repeated portal slices, and path optimization all beat generic forecasting. The best question was usually: what constraint is this market secretly obeying? Not: can I throw a more complicated model at the next tick?

The second pattern was that microstructure mattered more than theory alone. Option math helped, especially in Rounds 3 and 4, but execution decided whether the edge existed. Spreads, depth, fills, queue position, inventory limits, and portal horizon mattered as much as the theoretical relationship.

The third pattern was that the fill model was load-bearing. Without InterceptFill, a lot of local conclusions would have been wrong. Once the fill model matched the portal better, backtests and counterfactuals became much more meaningful.

The fourth pattern was that rejection was part of the edge. We rejected a lot: market-making models that did not match fills, OBI signals that failed sign checks, regime tilts that failed against the Round 2 baseline, option ideas that died after execution costs, Round 4 alpha stacking, Round 5 neural predictions, and naive market making on bad products. That mattered. Avoiding bad trades is part of building a good strategy.

My Role

My main role was strategy direction. I was not the main optimizer. I was the person pushing market hypotheses, weird mechanisms, and structural reads. I tried to move the search into better regions.

Round 2 is the clearest example. My OSMIUM handler became the dominant OSMIUM alpha and the foundation of the shipped R2 strategy. Dennis and the team did a lot of work after that, but they were optimizing inside a basin that my idea opened.

Dennis's role was different but equally important. He helped turn those ideas into tests, variants, backtests, counterfactuals, and actual submissions. The collaboration worked because I pushed direction and Dennis helped ground it.

The best summary is: I generated strategic direction, Dennis optimized and verified it, and the team's process turned the strongest ideas into submissions.

Final Takeaway & Architectural Stuff

Prosperity 4 was not a straight line from Round 1 to Round 5. It was a sequence of reframings.

Round 1 taught us the market mechanics. Round 2 showed that my OSMIUM mechanism could shift the entire strategy direction. Round 3 turned clue-reading into path optimization. Round 4 taught restraint when there were too many possible alphas. Round 5 proved that simple structure beats complex prediction when the data supports it.

The strongest part of the project was the loop: idea, implementation, replay, audit, ship, postmortem.

I drove a lot of the strategy inspiration. Dennis drove a lot of the optimization and verification. The team worked because those roles complemented each other.

Built a full-stack algorithmic trading research suite integrating strategy development, local simulation, portal-trace reconciliation, market microstructure analysis, derivatives-pricing research, statistical signal processing, and visualization tooling.

• Designed modular Python infrastructure for market-state data models, semicolon-delimited exchange-data ingestion, normalized order-book snapshots, fair-value estimation, execution simulation, strategy loading, parameter sweeps, grid search, artifact persistence, regression harnesses, and dashboard-based analysis.

• Built a full-stack algorithmic trading research suite integrating strategy development, local simulation, portal-trace reconciliation, market microstructure analysis, derivatives-pricing research, statistical signal processing, live-market backtesting, and visualization tooling.

• Designed modular Python infrastructure for market-state data models, semicolon-delimited exchange-data ingestion, normalized order-book snapshots, fair-value estimation, execution simulation, strategy loading, parameter sweeps, grid search, artifact persistence, regression harnesses, and dashboard-based analysis.

• Implemented a portal-calibrated, official-interface-compatible runtime with Trader.run(state), passive-order interception, late-taker flow, position-limit enforcement, queue-aware fill behavior, fresh-trade reconstruction, sticky market-trade handling, and tick-level order processing.

• Extended the simulator into a live-market backtesting engine for real-world trading data, supporting historical quote/trade replay, event-driven execution, exchange-style order matching, latency assumptions, transaction-cost modeling, slippage estimation, position/risk constraints, and strategy evaluation under realistic market microstructure conditions.

• Built latency-aware workflows for order-book reconstruction, quote/trade alignment, timestamp normalization, event sequencing, spread dynamics, wall-mid computation, passive-fill opportunity detection, markout analysis, and microstructure-noise filtering.

• Developed fair-value, execution, and derivatives analytics primitives including depth-weighted price estimates, microprice-style signals, spread-capture logic, order-flow imbalance, synthetic parity checks, smile residuals, Greeks estimation, volatility-surface diagnostics, no-arbitrage checks, and anomaly detection.

• Created reproducibility and diagnostics tooling: local run capture, portal submission replay, live-data replay logs, reconciliation scorecards, variant leaderboards, PnL curves, drawdown/risk reports, regression tests, artifact versioning, and Plotly/Dash/Qt dashboards for order books, fills, markouts, volatility surfaces, Greeks, residual heatmaps, and strategy diagnostics.